

4. Spectral Methods and Signal Processing [150 points]

In this computational exercise, use the Fast Fourier Transform (FFT) to analyze, differentiate, and translate a 1D wavepacket. Consider a modulated Gaussian pulse given by

$$f(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \cos(k_0 x)$$

where $\sigma = 1.0$ is the width of the envelope and $k_0 = 15.0$ is the carrier wavenumber. Below, analyze this function on the domain $x \in [-10, 10]$.

Write a Python code to perform the following tasks:

a) The Shannon-Nyquist sampling theorem states that to accurately resolve a signal, your sampling rate must be at least twice the highest frequency present in the signal.

- Create two discrete spatial grids for $x \in [-10, 10]$: a "fine" grid with $N = 512$ points, and a "coarse" grid with $N = 40$ points.
- Calculate the spatial step size Δx and the corresponding Nyquist wavenumber $k_{\text{Nyquist}} = \pi/\Delta x$ for both grids.
- Compute the FFT of $f(x)$ on both grids. Plot the magnitude of the Fourier transforms $|\tilde{f}(k)|$ versus the wavenumber k .
- *Question:* Based on your plots and your calculated k_{Nyquist} , explain why the FFT on the coarse grid fails to represent the true physical frequencies of the wavepacket. What is this phenomenon called?

b) The Fourier transform diagonalizes the derivative operator, such that $\mathcal{F}\{f'(x)\} = ik\tilde{f}(k)$. This allows for highly accurate "spectral" differentiation.

- Using your fine grid ($N = 512$), compute the numerical derivative of $f(x)$ by taking its FFT, multiplying by ik , and taking the Inverse FFT (IFFT). (*Hint: If you use standard FFT routines, ensure your k -array is ordered correctly, e.g., using `fftfreq` in `numpy`, to match the internal ordering of the FFT algorithm!*)
- Calculate the derivative using finite differences and the exact analytical derivative $f'_{\text{exact}}(x)$ using calculus.
- Plot your spectral derivative, the exact derivative and the finite difference derivative on the same graph to show they match.
- Plot the absolute error $|f'_{\text{spectral}}(x) - f'_{\text{exact}}(x)|$ and $|f'_{\text{finite_diff}}(x) - f'_{\text{exact}}(x)|$.

c) Suppose we want to translate the wavepacket by a distance $x_0 = 3.2$. If x_0 is not an exact integer multiple of Δx , shifting an array in real space requires messy interpolation. The Shift Theorem provides an elegant alternative.

- Determine and plot your computationally shifted signal alongside the exact shifted function $f_{\text{exact}}(x - 3.2)$ for the fine grid.

- *Question:* The FFT assumes periodic boundary conditions. What would happen to the wavepacket if you chose a shift of $x_0 = 15.0$? (You may test this in your code to verify your answer).

d) The "Fast" in FFT comes from the Cooley-Tukey algorithm, which reduces the computational complexity of the Discrete Fourier Transform (DFT) from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log_2 N)$. However, this efficiency depends heavily on the prime factorization of the array size N .

- Write a loop that measures the execution time of the 1D FFT for random arrays of size $N = 2^p$, where p ranges from 10 to 22 (i.e., $N \approx 10^3$ to 4×10^6).
- Plot the execution time T versus N on a log-log scale. On the same graph, plot a theoretical reference line $T_{ref} = c \cdot N \log_2(N)$ (choose a small constant c so the reference line aligns vertically with your data).
- Now, measure and print the FFT execution time for an array of size $N_1 = 2^{20} = 1,048,576$.
- Next, measure and print the FFT execution time for an array of size $N_2 = 1,000,003$ (which is a prime number).
- *Question:* Report the two times. Why is the FFT significantly slower for N_2 compared to N_1 , even though N_2 contains fewer data points?